Fase Residual sobre `Agent7250`: motivacion, implementacion, barridos de alpha_res y cierre metodologico

Proyecto EMG Prosthesis TD3

26 de marzo de 2026

1. Objetivo y continuidad de la linea

Este documento deja cerrada, de forma detallada, la fase de *Residual TD3* construida como continuacion directa del benchmark operativo del proyecto: `Agent7250`. Si en notas previas aparece la referencia informal a "Agent7200", el checkpoint correcto y congelado como base del trabajo es `Agent7250`.

La idea central de esta fase fue no volver a entrenar una politica completamente nueva ni volver a modificar reward, observacion o cuantizacion. Se busco una intervencion mas conservadora: partir de una politica ya buena y aprender solo una correccion pequena sobre ella. Esa decision fue coherente con la lectura acumulada del repo:

- `Agent7250` ya hacia buen tracking.
- Las ramas previas con regularizacion o warm-start tendian a politicas demasiado timidas.
- El piloto de accion alineada al actuador tendio al extremo opuesto: mas borde y mas saturacion.
- Hacia falta una forma de mejorar el compromiso tracking-esfuerzo sin destruir el prior ya aprendido por la politica base.

En la literatura, ese escenario encaja bien con *Residual Policy Learning*: mantener un controlador base y aprender una correccion pequena y dependiente del estado, en vez de reemplazar la politica completa [1, 2, 3]. Esa fue la justificacion conceptual de esta fase.

2. Punto de partida: benchmark congelado

La fase residual se construyo sobre el mismo baseline valido que ya venia usando el proyecto desde la corrida correcta post-fix del simulador. Se mantuvo sin cambios:

- observacion `markov52`,
- reward `trackingMseActionRateReward`,
- TD3 feedforward,
- remapeo y cuantizacion efectivos del actuador,
- entorno corregido y ya validado tras el bug del simulador.

Cuadro 1: Benchmark congelado usado como referencia durante toda la fase residual

Metrica	Valor
`trackingMSE`	0.043045
`trackingMAE`	0.160336
`actionL2`	0.596444
`saturationFraction`	0.392086
`deltaActionL2`	0.321385
`absPWM mean`	178.288566

Ese benchmark proviene de la linea de trabajo iniciada sobre TD3 continuo para protesis mioelectrica [4], en un contexto donde el control mioelectrico exige buen tracking pero tambien suavidad y estabilidad de accion [5]. La arquitectura base sigue siendo TD3, adecuada para accion continua y ya implementada de forma nativa en MATLAB Reinforcement Learning Toolbox [6, 7, 8].

3. Por que la fase residual era una idea razonable

La decision no fue arbitraria. Se apoyo en tres observaciones tecnicas del propio repositorio:

1. El benchmark ya estaba en una region util del espacio de politicas. No habia evidencia de que "empezar otra vez" fuera mejor que refinar una solucion buena.
2. Las intervenciones previas fallaban por exceso o por defecto: demasiado conservadoras o demasiado agresivas.
3. El problema ya no era aprender el movimiento base, sino corregir un compromiso fino entre tracking y agresividad.

Residual Policy Learning fue propuesto precisamente para estos escenarios: cuando existe un controlador o politica base razonable, y conviene aprender solo el termino faltante [1]. Trabajos posteriores en control continuo y autonomia compartida refuerzan la misma idea: una correccion pequena y estructurada puede ser mas estable y mas eficiente que reemplazar toda la politica [2]. En la misma direccion, lineas recientes de *fine-tuning* residual sobre politicas preentrenadas vuelven a destacar que la forma mas segura de mejorar una politica fuerte es mantener la base y aprender un residuo con magnitud controlada [3].

4. Que se implemento en el codigo

La fase residual no fue un cambio difuso; se codifico como una rama experimental clara y trazable dentro del proyecto.

Cuadro 2: Archivos clave de la fase residual y su funcion

Archivo	Funcion
`matlab_code/agents/agentTd3Residual7250.m`	Construye el nuevo agente residual a partir del checkpoint base `Agent7250`.
`matlab_code/agents/load_agent.m`	Incorpora el caso `td3_residual7250` al flujo normal de carga.
`matlab_code/src/loadSavedAgent.m`	Estandariza la carga del checkpoint base y de checkpoints residuales.
`matlab_code/src/run_agent7250_residual_policy_pilot.m`	Lanza el piloto principal con $\alpha = 0.20$.
`matlab_code/src/run_agent7250_residual_continuation.m`	Continua desde el mejor checkpoint residual para evaluar si mas episodios ayudan.
`matlab_code/src/run_agent7250_residual_alpha_sweep.m`	Ejecuta el barrido corto inicial $\alpha \in \{0.10, 0.20, 0.30\}$.
`matlab_code/src/run_agent7250_residual_alpha_refinement_sweep.m`	Ejecuta el refinamiento fino $\alpha \in \{0.18, 0.20, 0.22, 0.25\}$.
`matlab_code/src/reconstructResidualPolicyDiagnostics.m`	Reconstruye en postproceso la descomposicion entre accion base y correccion residual.
`matlab_code/src/@Env/step.m` y `saveEpisode.m`	Guardan estado, accion final y logs necesarios para diagnostico residual.
`matlab_code/src/summarizeEpisodeDirectory.m`	Agrega las metricas residuales por episodio y por corrida.

La implementacion se apoyo en componentes ya disponibles en MATLAB para TD3 y actores deterministas continuos basados en `dlnetwork`, asi como en capas funcionales para componer ramas dentro del actor [9, 10].

5. Que es exactamente alpha_res

La politica residual se definio como una composicion entre:

- una politica base congelada, $a_{\text{base}}(s)$, tomada de `Agent7250`,
- y una correccion aprendible, $a_{\text{res}}(s, a_{\text{base}})$.

La accion final se construyo como:

$$a(s) = \text{clip}\Big(a_{\text{base}}(s) + \alpha_{\text{res}}\, a_{\text{res}}(s, a_{\text{base}}), -1, 1\Big).$$

Aqui, α_{res} es el parametro mas importante de la fase:

- si α_{res} es pequeno, la correccion residual apenas puede mover la accion base;
- si α_{res} es grande, la politica residual gana libertad, pero tambien aumenta el riesgo de empujar demasiado al borde del actuador;
- por tanto, α_{res} controla el equilibrio entre *conservadurismo* y *capacidad real de mejora*.

Esta fue la razon de los barridos de α. No eran "barridos por probar algo"; eran la forma correcta de medir hasta donde convenia dejar separarse a la nueva politica respecto a `Agent7250`.

6. Protocolo experimental de la fase residual

6.1. Piloto residual principal

La primera corrida residual uso:

- checkpoint base: `Agent7250`,
- $\alpha_{\text{res}} = 0.20$,
- exploracion reducida: `0.02 / 0.002 / 1e-4`,
- `2000` episodios,
- guardado cada `50`,
- auditoria de cola con dos fases y ranking por tracking, saturacion, suavidad y esfuerzo.

La idea era deliberadamente conservadora: no se hizo sweep desde el inicio. Primero se quizo ver si una residual moderada podia dar una mejora defendible.

6.2. Continuacion del mejor residual

Una vez obtenido un candidato fuerte, se intento una continuacion mas larga sobre ese mismo residual para comprobar si habia ganancia adicional por mas episodios. Esto fue importante metodologicamente, porque el mejor checkpoint aparecio cerca del final de la corrida y habia duda razonable sobre si la cola seguia mejorando.

6.3. Barrido corto de alpha_res

Al confirmarse que la simple extension no mejoraba, se paso a un barrido corto:

$$\alpha_{\text{res}} \in \{0.10,\ 0.20,\ 0.30\}.$$

La logica fue:

- $\alpha = 0.10$: correccion muy conservadora;
- $\alpha = 0.20$: punto central del piloto;
- $\alpha = 0.30$: correccion mas expresiva, con riesgo de agresividad.

6.4. Refinamiento fino

Como el sweep corto ya sugeria que la mejor zona estaba alrededor de 0.20, se hizo una exploracion mas fina:

$$\alpha_{\text{res}} \in \{0.18,\ 0.20,\ 0.22,\ 0.25\}.$$

Ese refinamiento sirvio para responder una pregunta precisa: si realmente habia un punto mejor muy cerca de 0.20, o si la linea residual ya estaba localizada y madura.

7. Resultados del piloto residual

El piloto principal produjo el primer resultado claramente competitivo de toda la fase posterior a `Agent7250`. El mejor checkpoint auditado fue `Agent1850`; en auditoria completa ya aparecia como `ConditionA`, y en test visual de 50 simulaciones quedo como `ConditionB`.

Cuadro 3: Benchmark vs mejor residual del piloto principal

Metrica	`Agent7250`	Residual `Agent1850`
`trackingMSE`	0.043045	0.043445
`trackingMAE`	0.160336	0.163542
`actionL2`	0.596444	0.523597
`saturationFraction`	0.392086	0.272637
`deltaActionL2`	0.321385	0.299262

La lectura de ese resultado fue la razon por la que la linea residual se volvio la principal del proyecto:

- el tracking se mantenia practicamente igual, con una perdida de menos del 1 % en `trackingMSE`;
- la saturacion bajaba alrededor de un 30.5 %;
- el esfuerzo de control bajaba alrededor de un 12.2 %;
- la accion seguia siendo mas suave que la base.

Eso ya cumplia la `ConditionB`. Fue la primera mejora aceptable y defendible respecto al benchmark congelado.

8. Por que no se adopto “mas episodios” como respuesta

La continuacion fuerte del residual no confirmo una mejora adicional. El mejor checkpoint de esa extension quedo peor que el residual original en compromiso global.

Cuadro 4: Resultado de la continuacion fuerte del residual

Metrica	Mejor checkpoint de la continuacion
`trackingMSE`	0.045631
`actionL2`	0.598299
`saturationFraction`	0.383032
`deltaActionL2`	0.243518
`benchmarkStatus`	`Rejected`

La interpretacion correcta no fue “nunca mas entrenar”, sino algo mas fino: para esta configuracion residual concreta, seguir metiendo episodios ya no estaba mejorando el compromiso. La extension mejoro algo en suavidad local, pero degrado tracking y no produjo una politica mejor que el `Agent1850` del piloto.

9. Resultados del barrido corto de alpha_res

El primer barrido de α confirmo que el piloto no habia acertado por casualidad.

Cuadro 5: Barrido corto inicial de α_{res}

α_{res}	trackingMSE	actionL2	saturationFraction	deltaActionL2	Estado final
0.10	0.042506	0.643112	0.498572	0.263525	Rejected
0.20	0.043445	0.523597	0.272637	0.299262	ConditionB
0.30	0.044501	0.533454	0.301742	0.325751	Rejected

La tabla muestra bien que es α en la practica:

- $\alpha = 0.10$ fue demasiado pequeno: la residual casi no corrijio de forma util y aun asi termino con mas saturacion y mas esfuerzo de lo deseado.
- $\alpha = 0.30$ fue demasiado grande: dio una residual demasiado expresiva, que ya no preservaba bien el compromiso tracking-agresividad.
- $\alpha = 0.20$ quedo en medio: suficiente para corregir a la base, pero sin volverse inestable.

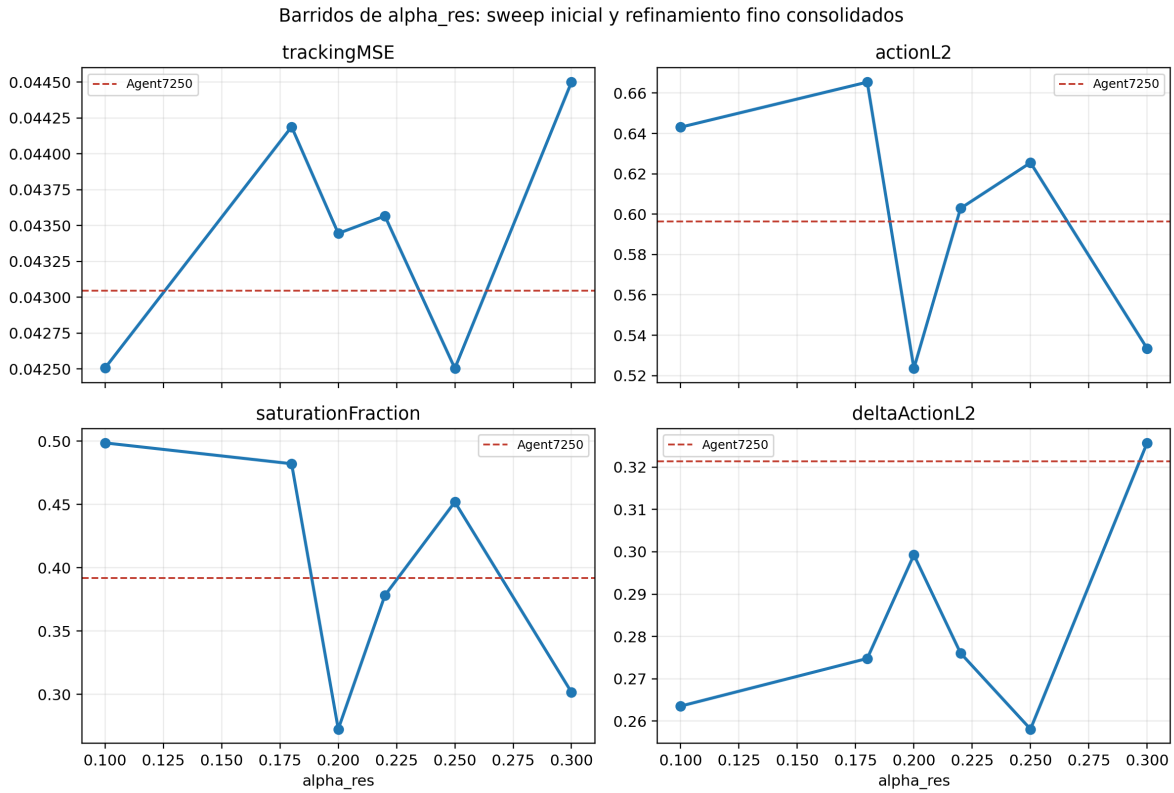

Figura 1: Metricas finales de los barridos de α_{res}. Las lineas rojas discontinuas marcan el benchmark `Agent7250`.

10. Resultados del refinamiento fino

Una vez localizado 0.20 como zona fuerte, el refinamiento fino respondio si habia una mejora pequena y no obvia alrededor de ese valor.

Cuadro 6: Refinamiento fino de α_{res}

α_{res}	trackingMSE	actionL2	saturationFraction	deltaActionL2	Estado final
0.18	0.044189	0.665430	0.482118	0.274781	Rejected
0.20	0.043445	0.523597	0.272637	0.299262	ConditionB
0.22	0.043566	0.602889	0.378192	0.276052	Rejected
0.25	0.042503	0.625515	0.451917	0.258078	Rejected

El refinamiento cerro la linea con bastante claridad:

- $\alpha = 0.18$ no fue una version "mas segura"; al contrario, salio peor en esfuerzo y saturacion.
- $\alpha = 0.22$ quedo cerca en tracking, pero ya perdia demasiada eficiencia en control.
- $\alpha = 0.25$ compro tracking a costa de mas borde y mas accion.
- $\alpha = 0.20$ siguio siendo el unico punto que sostuvo el compromiso completo.

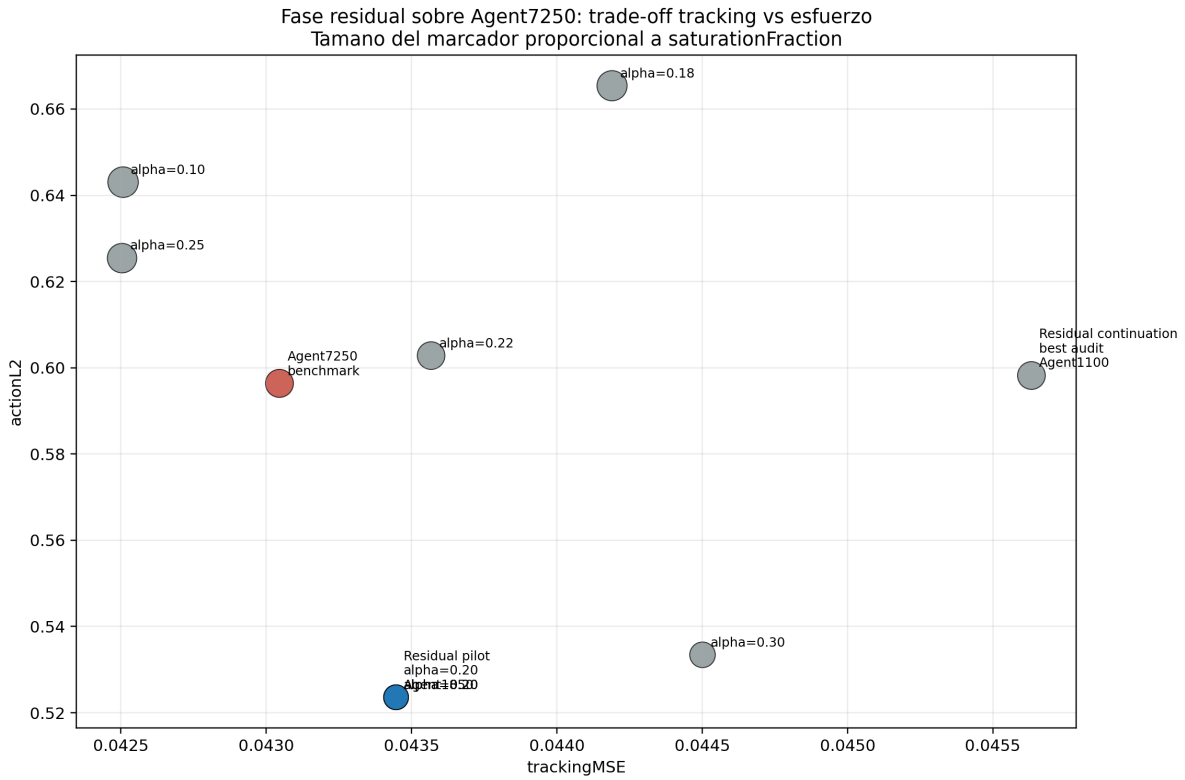

Figura 2: Panorama completo de la fase residual. El tamano del marcador es proporcional a `saturationFraction`. El mejor candidato final sigue siendo el residual con $\alpha = 0.20$.

11. Que dicen los diagnosticos residuales

La ventaja de esta fase no fue solo encontrar un mejor agente; tambien permitio entender *como* se aparta la politica residual respecto a la base.

Durante la auditoria se resumieron, entre otras, estas metricas:

- `auditResidualActionL2Mean`: magnitud media de la correccion residual.

- `auditBaseToFinalActionDeltaMean`: cuanto se mueve la accion final respecto a la base.
- `auditSignOverrideFractionMean`: con que frecuencia la residual cambia el signo de la accion base.

En el mejor caso, $\alpha = 0.20$, esos valores fueron moderados:

- `ResidualActionL2Mean` ≈ 0.0452,
- `BaseToFinalActionDeltaMean` ≈ 0.0850,
- `SignOverrideFractionMean` ≈ 0.0981.

Eso es precisamente lo que se queria de una residual bien comportada: no colapsar a una politica totalmente distinta, sino corregir la base lo suficiente para rebajar agresividad sin perder casi todo el tracking.

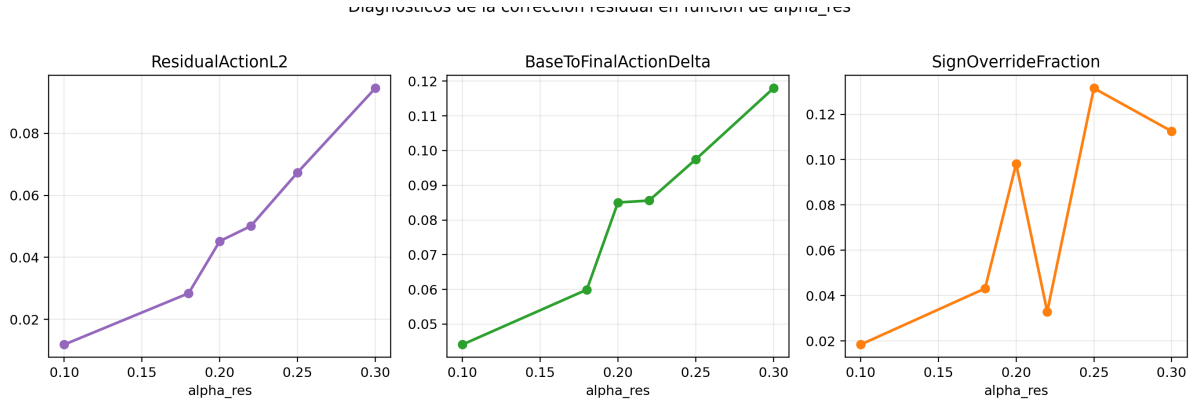

Figura 3: Diagnosticos de la correccion residual en funcion de α_{res}. A mayor α, la residual gana capacidad para apartarse de la base; el punto util quedo en la zona intermedia.

12. Interpretacion final de la fase

La fase residual deja una conclusion mas fuerte que un simple "este agente fue mejor":

1. La idea residual estaba bien planteada. A diferencia de otras lineas recientes, si produjo una mejora real y defendible.
2. El mejor candidato actual del proyecto en esta familia es el residual `Agent1850` con $\alpha_{\text{res}} = 0.20$.
3. Entrenar mas, con la misma configuracion, no mejoro la politica.
4. Barrer α tampoco encontro un punto mejor que 0.20, ni en el sweep corto ni en el refinamiento fino.
5. Por tanto, la linea residual no solo produjo una mejora: tambien quedo suficientemente localizada y cerrada desde el punto de vista metodologico.

La leccion clave es que la mejora no vino de "hacer otra reward" ni de "meter mas episodios", sino de reorganizar la accion aprendida alrededor de una politica base fuerte. Ese es exactamente el tipo de escenario donde las ideas residuales son utiles [1, 2, 3].

13. Conclusion operativa

Si el proyecto necesita fijar un candidato actual para entrega o documentacion final, la recomendacion tecnica ya es clara:

- benchmark historico congelado: `Agent7250`,
- mejor mejora encontrada sobre esa linea: residual `Agent1850` con $\alpha_{\text{res}} = 0.20$,
- estado final frente al benchmark: `ConditionB`.

En otras palabras, la linea de trabajo que continuo realmente a `Agent7250` y aporto una mejora defendible fue la residual. Los barridos posteriores no la reemplazaron; la confirmaron.

Referencias

[1] Tom Silver, Kelsey Allen, Joshua B. Tenenbaum, and Leslie P. Kaelbling. Residual policy learning. https://arxiv.org/abs/1812.06298, 2018. Accessed: 2026-03-26.

[2] Charles Schaff and Matthew R. Walter. Residual policy learning for shared autonomy. https://arxiv.org/abs/2004.05097, 2020. Accessed: 2026-03-26.

[3] Residual Off-Policy RL Project Page. Introducing resfit: Residual fine-tuning with off-policy rl. https://residual-offpolicy-rl.github.io/, 2025. Accessed: 2026-03-26.

[4] Jose Gabriel Fuertes Bustos, Carlos Ayala, Lorena Isabel Barona, Marco E. Benalcazar, and Angel Leonardo Valdivieso. Continuous operation of a myoelectric hand prosthesis using the td3 reinforcement learning algorithm. Technical manuscript provided by the user, 2026. Baseline conceptual article used as historical reference for the project.

[5] Ziming Chen, Huasong Min, Dong Wang, Ziwei Xia, Fuchun Sun, and Bin Fang. A review of myoelectric control for prosthetic hand manipulation. *Biomimetics*, 8(3):328, 2023.

[6] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. https://arxiv.org/abs/1802.09477, 2018. TD3 original paper.

[7] MathWorks. rltd3agent. https://www.mathworks.com/help/reinforcement-learning/ref/rl.agent.rltd3agent.html, 2026. Accessed: 2026-03-12.

[8] MathWorks. rltd3agentoptions. https://www.mathworks.com/help/reinforcement-learning/ref/rl.option.rltd3agentoptions.html, 2026. Accessed: 2026-03-12.

[9] MathWorks. rlcontinuousdeterministicactor. https://www.mathworks.com/help/reinforcement-learning/ref/rl.function.rlcontinuousdeterministicactor.html, 2026. Accessed: 2026-03-26.

[10] MathWorks. functionlayer. https://www.mathworks.com/help/deeplearning/ref/nnet.cnn.layer.functionlayer.html, 2026. Accessed: 2026-03-26.